

Computer Assignment 3: Identifying Flower Species Using Convolutional Neural Networks

Technical Analysis of the Python Implementation

A source-faithful report for a research-oriented software portfolio

Contents

Abstract	2
1 Introduction	2
2 Project Overview	2
3 Technical Foundations	3
3.1 Supervised multiclass image classification	3
3.2 Convolutional representation learning	4
3.3 ReLU activation and batch normalization	4
3.4 Softmax output	4
4 Data and Input Processing	4
4.1 Dataset acquisition	4
4.2 Training and validation split	4
4.3 On-the-fly augmentation	5
4.4 Normalization	5
5 System Architecture and Code Organization	5
6 Detailed Methodology	6
6.1 Model input and feature extractor	6
6.2 Global average pooling and dense head	7
6.3 L2 regularization and dropout	7
7 Objective Function and Mathematical Formulation	7
8 Optimization and Hyperparameter Search	8
8.1 Optimizer behavior in the implementation	9
9 Full Training Procedure and Callbacks	9
10 Detailed Evaluation Pipeline	10
11 Inference Demonstration	10
12 Implementation Details	11

12.1 Configuration	11
12.2 Filesystem and artifact management	11
12.3 Runtime hardware awareness	11
13 Execution Workflow	11
14 Design Decisions and Rationale	12
15 Computational Considerations	12
16 Reproducibility and Reliability	13
17 Limitations and Technical Considerations	13
18 Overall Technical Assessment	14
19 Conclusion	15
Source Traceability Notes	15

Abstract

The uploaded Python source implements an end-to-end five-class image-classification workflow based on a regularized convolutional neural network (CNN). The program downloads the TensorFlow flower-photo dataset, constructs deterministic training and validation pipelines, performs on-the-fly image augmentation and normalization, builds a four-block CNN with batch normalization and L2 regularization, compares four optimizers over three learning rates, selects a configuration using peak validation accuracy, retrain the selected architecture with checkpointing, learning-rate reduction, and early stopping, and computes accuracy, macro-averaged precision/recall/F1, weighted precision/recall/F1, classification reports, and confusion matrices. It also performs a small inference demonstration and packages generated artifacts into a ZIP archive. The analysis below focuses on what the implementation actually does, how its mathematical and computational components fit together, and what can and cannot be concluded from the source alone. No experimental result values are fabricated in this report; numerical outcomes are treated as runtime outputs of the program rather than as source-level facts.

1 Introduction

The project is a supervised multiclass image-classification system whose explicit task is to distinguish five flower categories: daisy, dandelion, roses, sunflowers, and tulips. The source identifies the work as “Computer Assignment 3 (CA3): Identifying Flower Species Using CNNs” and explicitly lists hyperparameter selection, regularization, training-accuracy reporting, multiclass evaluation, convergence-curve generation, optimizer comparison, sample inference, and artifact packaging as assignment objectives.

The implementation is more than a single model definition. It is a compact experiment pipeline. It performs data acquisition, dataset splitting, input-pipeline optimization, model construction, controlled hyperparameter comparison, full training of a selected model, post-training evaluation, inference, and output packaging in one executable script. The code is organized primarily as a sequence of top-level stages supported by a small set of reusable functions rather than as a multi-module software package.

This report therefore treats the source as both a machine-learning experiment and a software artifact. The first perspective explains the learning problem, model, objective function, metrics, and optimization procedure. The second explains how paths, configuration values, functions, callbacks, TensorFlow datasets, and generated files interact. Throughout, the report distinguishes three levels of interpretation: the behavior explicitly implemented in the source, the established meaning of the underlying method, and reasonable implications of the implementation. Where the author’s intention cannot be proven from the code, the discussion is phrased accordingly.

2 Project Overview

At the highest level, execution follows the sequence below.

1. Define global configuration values and initialize reproducibility settings.

2. Download and unpack the five-class flower dataset, then create an 80/20 training-validation split.
3. Cache and prefetch the TensorFlow datasets to improve input throughput.
4. Save a sample-image visualization for basic dataset inspection.
5. Construct the regularized CNN architecture and compile it with a selectable optimizer and learning rate.
6. Train twelve short configurations formed by four optimizers and three learning rates.
7. Select the configuration with the highest observed validation accuracy.
8. Train a fresh instance of that configuration with checkpointing, adaptive learning-rate reduction, and early stopping.
9. Reload the best checkpoint and evaluate the model on both training and validation datasets.
10. Compute class-aggregated metrics and classification reports, and generate confusion matrices.
11. Run inference on a small set of images selected from the dataset directory structure.
12. Write textual and tabular artifacts and package the resulting output directory into a ZIP archive.

The pipeline is fully executable from the script as written, subject to availability of its external dependencies and the remote dataset URL. There is no separate command-line interface, configuration file, test suite, or application entry-point function. Instead, execution begins during import/run time as soon as the top-level statements are evaluated.

3 Technical Foundations

3.1 Supervised multiclass image classification

Each image is assigned one of $K = 5$ discrete class labels. The network maps an input image x to a probability vector

$$\hat{\mathbf{p}} = f_{\theta}(x) \in [0, 1]^K, \quad \sum_{k=1}^K \hat{p}_k = 1, \quad (1)$$

where θ denotes the trainable model parameters. The predicted class is the index of the largest component of $\hat{\mathbf{p}}$.

The implementation uses integer class labels produced by `image_dataset_from_directory` and compiles the model with `SparseCategoricalCrossentropy`. This is appropriate for mutually exclusive multiclass classification because each training example has one integer-valued target rather than a one-hot target vector.

3.2 Convolutional representation learning

A two-dimensional convolution applies a learned kernel to local spatial neighborhoods of the image. Abstractly, for an output channel c , the pre-activation at spatial location (i, j) can be written as

$$z_{i,j,c} = \sum_{u,v,d} W_{u,v,d,c} x_{i+u,j+v,d} + b_c, \quad (2)$$

with subsequent nonlinear processing. The source uses 3×3 kernels and **same** padding in each convolutional block. The number of filters increases from 32 to 64, then 128, then 256, allowing progressively richer feature representations while max pooling reduces spatial resolution.

3.3 ReLU activation and batch normalization

The convolutional and dense hidden representations use the rectified linear unit

$$\text{ReLU}(z) = \max(0, z). \quad (3)$$

Batch normalization is placed after every convolution and after the dense feature layer. In general form, normalization acts on a minibatch using an estimated mean and variance followed by learned scale and shift parameters. In this implementation it is therefore part of the repeated pattern “convolution $\rightarrow$ batch normalization $\rightarrow$ ReLU $\rightarrow$ max pooling” and of the dense feature head “dense $\rightarrow$ batch normalization $\rightarrow$ ReLU $\rightarrow$ dropout”.

3.4 Softmax output

The final dense layer produces five class scores and applies softmax:

$$\hat{p}_k = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)}. \quad (4)$$

The resulting vector is interpreted as class probabilities by the inference and metric code. Because the output layer already applies softmax, the loss is configured with `from_logits=False`.

4 Data and Input Processing

4.1 Dataset acquisition

The function `load_and_prepare_dataset` downloads the TensorFlow example flower dataset from a fixed Google Cloud Storage URL and asks `tf.keras.utils.get_file` to untar it into a cache directory under the project’s output directory. The code then counts JPEG files recursively through class subdirectories and prints the total number found. The five class names are recovered from `train_ds.class_names` rather than hard-coded into the classification model.

4.2 Training and validation split

Two calls to `image_dataset_from_directory` create complementary subsets from the same directory tree. Both calls use `validation_split=0.2`, the same random seed, image size $180 \times$

180, and batch size 32. One call requests the “training” subset and the other requests the “validation” subset. Thus the intended partition is 80% training and 20% validation.

The training dataset is shuffled and the validation dataset is not. After construction, the script applies TensorFlow data-pipeline optimization:

$$\text{training pipeline} = \text{prefetch}(\text{shuffle}(\text{cache}(D_{\text{train}}))), \quad (5)$$

while the validation pipeline is cached and prefetched without an additional shuffle. The exact code is `train_ds_raw.cache().shuffle(1000, seed=RANDOM_SEED).prefetch(buffer_size=AUTOTUNE)` and the analogous validation expression.

Caching is particularly relevant here because the dataset can be reused for multiple optimizer-learning-rate experiments. Prefetching allows TensorFlow to overlap input preparation with model execution. These optimizations target throughput; they do not alter the conceptual learning objective.

4.3 On-the-fly augmentation

Augmentation is embedded inside the model rather than applied when the dataset is written to disk. Four stochastic transformations are configured:

- horizontal flipping;
- random rotation with factor 0.15;
- random zoom with factor 0.15;
- random contrast with factor 0.1.

The augmentation stage is therefore part of the forward pipeline presented to the network during training. This design avoids creating a second stored copy of the dataset and lets the model receive different transformed views across training iterations.

4.4 Normalization

Pixel values are rescaled by $1/255$, producing values in the approximate interval $[0, 1]$ for the expected 8-bit image inputs:

$$x_{\text{norm}} = \frac{x}{255}. \quad (6)$$

The normalization is explicit in the model through `layers.Rescaling`. Because the same model is used for training and inference, this also reduces the chance that a separate inference script will forget the preprocessing step.

5 System Architecture and Code Organization

The implementation contains the following functional units.

There are no user-defined classes. The model is assembled with the Keras Functional API and returned as a compiled `keras.Model`. Most experiment coordination remains at top level. This

Function	Responsibility	Source anchor
<code>set_seed</code>	Synchronizes Python, NumPy, and TensorFlow random seeds	lines 71–78
<code>load_and_prepare_dataset</code>	Downloads, validates, splits, and labels the dataset	lines 94–147
<code>plot_dataset_samples</code>	Saves a nine-image sample visualization	lines 159–174
<code>build_regularized_cnn</code>	Constructs and compiles the CNN for a chosen optimizer/LR	lines 206–370
<code>extract_ground_truth_and_probabilities</code>	Collects labels, predictions, and probabilities	lines 545–558
<code>demonstrate_inference_on_samples</code>	Implements prediction on selected files and records results	lines 658–780

Table 1: Primary reusable functions in the uploaded source.

gives the script a linear, assignment-oriented structure that is easy to execute from start to finish, although it also means that concerns such as configuration, experiment orchestration, evaluation, and packaging are not separated into independent modules.

One source-level detail is worth documenting explicitly: `build_regularized_cnn` is defined twice, first at lines 181–199 and again at lines 206–224. The second definition is the one that remains bound to the function name when execution reaches the first call at line 374. The two blocks have the same documented purpose, and the later definition contains the actual implementation used for model construction. This is best understood as a redundant definition left by the source transformation rather than as two distinct architectures.

6 Detailed Methodology

6.1 Model input and feature extractor

The model begins with an input tensor of shape

$$(180, 180, 3), \quad (7)$$

corresponding to RGB images. The first layers apply augmentation followed by rescaling. The convolutional backbone then consists of four blocks.

Block	Filters	Kernel	Padding	Pooling
1	32	3×3	same	2×2
2	64	3×3	same	2×2
3	128	3×3	same	2×2
4	256	3×3	same	2×2

Table 2: Convolutional backbone specified by the implementation.

The convolutional layers all carry an L2 kernel regularizer. Each convolution is followed by batch normalization and ReLU, and each block ends with max pooling. Ignoring boundary details, four pooling operations reduce the spatial dimensions from 180×180 to approximately 11×11 , while the channel depth grows from 3 to 256. Consequently, the representation becomes lower-resolution but higher-dimensional as depth increases.

6.2 Global average pooling and dense head

Instead of flattening the final feature map into a potentially large vector, the implementation uses global average pooling. For a feature map $F \in \mathbb{R}^{H \times W \times C}$, the pooled channel representation is

$$g_c = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W F_{i,j,c}. \quad (8)$$

This produces one scalar per channel and therefore converts the final convolutional representation into a compact vector with 256 features. The dense head then applies a 256-unit dense layer with L2 regularization, batch normalization, ReLU, and dropout with rate 0.3, followed by the five-unit softmax classifier.

This head is computationally lighter than a flattening layer followed by a large fully connected matrix. It also makes the classifier less directly dependent on an exact spatial grid while preserving channel-wise information.

6.3 L2 regularization and dropout

For a set of weights W , L2 regularization contributes a term proportional to

$$\lambda \|W\|_2^2 \quad (9)$$

to the training objective. In the source, $\lambda = 10^{-4}$ is passed to the convolutional and dense kernel regularizers. The implementation therefore penalizes large kernel values in the learned feature extractor and dense feature projection.

Dropout is configured with probability $p = 0.3$ on the dense representation. During training, units are randomly omitted according to the dropout mechanism; during inference, the layer behaves deterministically. The source describes the purpose in terms of reducing co-adaptation of hidden features. This makes dropout complementary to data augmentation and L2 regularization rather than a replacement for them.

7 Objective Function and Mathematical Formulation

The model is compiled with sparse categorical cross-entropy. For a sample with true class y and predicted probabilities $\hat{\mathbf{p}}$, the loss can be expressed as

$$\ell(\theta; x, y) = -\log \hat{p}_y, \quad (10)$$

where $\hat{p}_y$ is the probability assigned to the correct class. Over a minibatch of size N , the mean data loss is

$$\mathcal{L}_{\text{CE}}(\theta) = -\frac{1}{N} \sum_{i=1}^N \log p_{\theta}(y_i | x_i). \quad (11)$$

Because L2 kernel regularizers are attached to multiple layers, the effective objective optimized by Keras has the conceptual form

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{CE}}(\theta) + \sum_{r \in \mathcal{R}} \lambda \|W_r\|_2^2, \quad (12)$$

where $\mathcal{R}$ denotes the regularized kernels and $\lambda = 10^{-4}$. The exact internal aggregation and scaling of regularizer contributions are governed by the Keras implementation; the source itself does not manually compute this objective.

Classification accuracy is calculated as the proportion of correct class predictions:

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\hat{y}_i = y_i). \quad (13)$$

For per-class evaluation, let TP_k , FP_k , and FN_k denote the true positives, false positives, and false negatives associated with class k . Then

$$P_k = \frac{TP_k}{TP_k + FP_k}, \quad (14)$$

$$R_k = \frac{TP_k}{TP_k + FN_k}, \quad (15)$$

$$F1_k = \frac{2P_k R_k}{P_k + R_k}. \quad (16)$$

The code obtains macro averages and weighted averages through scikit-learn. Macro averaging gives each class equal weight, while weighted averaging weights each class by its support. The source also requests `zero_division=0`, avoiding undefined metric exceptions for a class with no predicted positive instances in the relevant calculation.

8 Optimization and Hyperparameter Search

The program compares four optimizers:

- Adam;
- stochastic gradient descent with momentum and Nesterov acceleration;
- RMSprop;
- Adagrad.

Three learning rates are evaluated: 10^{-2} , 10^{-3} , and 10^{-4} . The Cartesian product creates twelve configurations. Each configuration is trained for 15 epochs using the same training and validation pipelines. Before each trial, `set_seed` is called and a fresh model is constructed.

The model selection criterion is the maximum validation accuracy attained at any epoch in the short trial:

$$\text{Score}(c) = \max_{t \in \{1, \dots, 15\}} \text{ValAcc}_{c,t}, \quad (17)$$

where c indexes a candidate optimizer-learning-rate pair. The row with the largest score is selected. The source then stores its optimizer name and learning rate as `BEST_OPTIMIZER` and `BEST_LEARNING_RATE`.

This is a simple and transparent search strategy, but it is important to interpret it correctly. The selection criterion is not the final validation accuracy at epoch 15 and not a statistically independent test score. It is the best validation accuracy observed during the short search

run. Furthermore, the same validation split is subsequently used for full-model early stopping, checkpoint selection, and final validation evaluation. This is not inherently an execution bug, but it means the reported validation metrics should be understood as model-selection metrics rather than as an untouched external test estimate.

The source also exposes an `adamw` option inside `build_regularized_cnn`. However, `adamw` is not part of the grid-search candidate list, so that branch is available as a model-building option but does not participate in the comparative study described by the loop.

8.1 Optimizer behavior in the implementation

The implementation uses the standard TensorFlow/Keras optimizer classes with a configurable learning rate. For the momentum SGD branch, momentum is fixed at 0.9 and Nesterov acceleration is enabled. Adam, RMSprop, and Adagrad use only the supplied learning rate in the constructor. The source does not independently implement optimizer update equations; it delegates them to Keras.

The practical role of the learning rate is to control the scale of parameter updates. The grid therefore examines whether the different optimizer update rules respond better to comparatively large, medium, or small learning-rate settings under a common CNN architecture and regularization configuration.

9 Full Training Procedure and Callbacks

After selecting the best search configuration, the script creates a new model and trains it for up to 25 epochs. Three callbacks govern this stage.

Callback	Configuration	Role
ModelCheckpoint	monitor <code>val_accuracy</code> , save best	Preserves the checkpoint with the highest observed validation accuracy.
ReduceLROnPlateau	monitor <code>val_loss</code> , factor 0.5, patience 4, minimum 10^{-6}	Reduces the learning rate after a period without validation-loss improvement.
EarlyStopping	monitor <code>val_accuracy</code> , patience 10, restore best weights	Stops training after sustained stagnation in validation accuracy and restores the best monitored weights.

Table 3: Training callbacks specified by the source.

The use of both checkpointing and early stopping creates two related notions of “best”: the checkpoint file tracks the best validation-accuracy model seen during training, while early stopping restores the best weights according to the same monitored metric. The later explicit reload from the checkpoint makes the evaluation stage robust to the exact state left in memory after training.

10 Detailed Evaluation Pipeline

The function `extract_ground_truth_and_predictions` iterates through a dataset and calls `model.predict` on each batch. It collects three arrays: true labels, predicted class indices, and full probability vectors. The predicted class is the index of the maximum probability, implemented with `argmax`.

The training and validation datasets are both evaluated with the same selected model. The script computes accuracy directly using the equality between true and predicted labels. It then obtains macro and weighted precision, recall, and F1-score with scikit-learn’s `precision_recall_fscore_support`. Separate classification reports are generated for the training and validation partitions and saved as CSV files.

The confusion matrices are computed as

$$C_{ij} = \#\{n : y_n = i, \hat{y}_n = j\}, \quad (18)$$

so that each row corresponds to a true class and each column to a predicted class. The source visualizes these matrices with Seaborn but, per the scope of this technical report, the visual outputs are not reproduced here.

The evaluation design is intentionally richer than accuracy alone. Macro-averaged metrics make class-level performance visible even when class frequencies differ, while weighted metrics incorporate support. Confusion matrices additionally expose which specific class pairs are most frequently confused. These are appropriate diagnostics for a five-class classification problem because two models with similar overall accuracy can have materially different per-class behavior.

11 Inference Demonstration

The inference stage selects a small sample of image files directly from the dataset’s class directories. For each class name, it lists supported image extensions and selects the lexicographically last path returned by sorting. It stops after the requested number of samples, with a recursive search fallback if no class-specific images can be selected.

Each selected image is resized to 180×180 , converted to an array, expanded to a batch dimension, and passed through the trained model. The class with maximum predicted probability becomes the predicted label:

$$\hat{y} = \arg \max_{k \in \{1, \dots, K\}} \hat{p}_k. \quad (19)$$

The associated confidence value is the corresponding softmax probability multiplied by 100. The function also infers an “actual” class from the parent directory name and records image name, actual class, predicted class, and confidence percentage in a CSV table.

A subtle but important point is that this demonstration does not constitute an independent test set. The selected files are drawn from the same dataset directory structure that underlies the training-validation split. Therefore, the inference stage is best interpreted as an operational demonstration of the prediction interface rather than as a new generalization benchmark.

12 Implementation Details

12.1 Configuration

The core configuration is centralized near the top of the source: random seed 123, image size 180×180 , three channels, batch size 32, five target classes, 25 maximum full-training epochs, and 15 comparison epochs. Output paths are also centralized under `ca3_assignment_outputs`, with subdirectories for plots, tables, and saved models.

This is a good software-engineering choice for a self-contained assignment script because key parameters can be inspected and modified without searching through the entire file. At the same time, the configuration is not represented by a structured object or external configuration file, so experiments are not automatically recorded as versioned configurations.

12.2 Filesystem and artifact management

The script creates output directories with `makedirs(parents=True, exist_ok=True)`. It optionally copies the downloaded archive to the output directory, writes CSV summaries, writes a text assignment report, saves the best model checkpoint in Keras format, and finally constructs a ZIP archive from the output directory while excluding the internal dataset cache.

The packaging logic computes relative paths and stores them inside the archive under a `ca3_results` prefix. This gives the project a reproducible artifact boundary: model files, tables, plots, and summaries can be transferred together without embedding the internal cache directory.

12.3 Runtime hardware awareness

The source prints the installed TensorFlow version and checks for physical GPU devices. If no GPU is found, it states that the script is running on CPU. This is useful operational diagnostics, but the code does not set a device explicitly, configure GPU memory growth, or attempt to enforce deterministic GPU kernels.

13 Execution Workflow

The complete execution can be read as a dependency chain.

1. Global constants are defined, directories are created, and random generators are seeded.
2. Dataset acquisition returns raw training and validation datasets, class names, and the dataset path.
3. The raw datasets are converted to cached, shuffled/prefetched TensorFlow pipelines.
4. A sample visualization is generated for inspection.
5. The CNN constructor creates a compiled model with the requested optimizer and learning rate.

6. Twelve short training runs are executed, each beginning from a freshly constructed model after reseeding.
7. A pandas DataFrame stores one record per search configuration, including final and peak validation statistics.
8. The best configuration is extracted and a new model is instantiated.
9. Full training applies checkpointing, learning-rate reduction, and early stopping.
10. The best checkpoint is reloaded, then both datasets are evaluated in a second pass.
11. Aggregate metrics and per-class reports are written to CSV files; confusion matrices are generated.
12. Sample image inference writes a final CSV table.
13. A human-readable assignment summary is saved, and all generated artifacts are packaged into a ZIP archive.

The ordering matters. For example, evaluation depends on the existence of the selected model checkpoint, and inference depends on both the trained model and the dataset directory. Likewise, the final package is produced only after all earlier artifacts exist.

14 Design Decisions and Rationale

Several design choices are strongly supported by the implementation itself.

First, preprocessing is integrated into the model. This makes the training and inference path conceptually unified: the same model object knows how to augment and rescale inputs. Second, global average pooling is used instead of flattening the final feature map, limiting the dimensionality of the dense head. Third, regularization is layered rather than singular: augmentation, batch normalization, L2 penalties, dropout, early stopping, and learning-rate reduction all play different roles.

The optimizer comparison is also structured as a controlled grid. The CNN architecture, batch size, augmentation, regularization coefficient, and trial length remain constant while optimizer and learning rate vary. This does not constitute an exhaustive hyperparameter search, but it gives the assignment a clear comparative design.

It is more difficult to infer motivations for some choices. For example, the exact values 0.15 for rotation and zoom, 0.1 for contrast, 10^{-4} for L2, and 0.3 for dropout are explicit settings, but the source does not explain why those values were chosen. The report therefore treats them as implementation parameters rather than attributing an undocumented rationale to the author.

15 Computational Considerations

Let an input batch contain N images and let the spatial dimensions before pooling be $H \times W$. A dense operation with d_{in} inputs and d_{out} units costs approximately $O(d_{\text{in}}d_{\text{out}})$ per batch

element. A convolution with kernel size $k \times k$, input channels C_{in} , output channels C_{out} , and spatial size $H \times W$ has a leading arithmetic cost proportional to

$$O(HWk^2C_{\text{in}}C_{\text{out}}) \quad (20)$$

per image, ignoring constant factors and implementation-specific acceleration.

The first convolution operates on a large spatial grid but few channels; later convolutions operate on smaller spatial grids but substantially more channels. The four pooling operations reduce spatial work as channel width grows. Global average pooling also prevents the classifier head from multiplying a flattened $11 \times 11 \times 256$ tensor by a large dense matrix.

The dominant practical cost of the script is not a single training pass but the experiment multiplicity: twelve grid-search runs at up to 15 epochs, followed by an additional full training run of up to 25 epochs. Consequently, runtime depends strongly on whether TensorFlow uses a GPU, which the script reports but does not enforce.

Caching can reduce repeated disk reads across the many search runs, but it increases memory use because cached tensors are retained by the input pipeline. The appropriateness of this trade-off depends on the runtime environment and dataset size.

16 Reproducibility and Reliability

The code makes several concrete reproducibility efforts. The seed 123 is applied to Python's `random` module, NumPy, TensorFlow, and the `PYTHONHASHSEED` environment variable. The dataset split uses that same seed. Augmentation layers and dropout are also given the seed value.

These settings improve repeatability, but they do not establish absolute bit-for-bit determinism across all hardware and software configurations. The source does not enable a global deterministic-operation mode, pin package versions, record the exact TensorFlow/Keras/scikit-learn versions to a manifest, or store the full experiment configuration in machine-readable form. Therefore, the appropriate claim is that the implementation is seed-controlled rather than fully environment-reproducible.

A further reliability consideration concerns evaluation protocol. The validation set is involved in three distinct decisions: optimizer-learning-rate selection, checkpoint selection, and early-stopping weight restoration. The final validation metrics are therefore informative about performance under the implemented selection process, but they should not be interpreted as an unbiased estimate from a completely untouched test set. A stronger research protocol would reserve a separate test partition or use nested validation for model selection.

17 Limitations and Technical Considerations

The first limitation is structural redundancy: the CNN builder is defined twice. The later definition is effective, so the duplicate does not change the executed architecture, but it makes the source less maintainable and could confuse future readers.

Second, the assignment is packaged as a single script. This is convenient for coursework but gives less separation of concerns than a research software project might normally use. Dataset

handling, model construction, experiment orchestration, evaluation, inference, and packaging are coordinated in one file.

Third, the hyperparameter search is narrow. Only optimizer type and learning rate are varied, while architecture, dropout rate, L2 strength, augmentation parameters, batch size, and image size remain fixed. Therefore, the search identifies the best configuration among the twelve tested combinations rather than the globally best model under all plausible hyperparameters.

Fourth, the validation set is reused throughout model development. As discussed above, this affects how final validation metrics should be interpreted. There is no completely separate test set in the source.

Fifth, the sample inference routine chooses files from the same dataset directory rather than a separately maintained external test collection. This is useful as a demonstration but weak as a generalization test.

Sixth, the artifact pipeline is output-oriented but not accompanied by metadata that records the complete software environment. The ZIP file contains generated outputs, but the source does not write a dependency lockfile, operating-system identifier, GPU model, or package-version snapshot into the artifact directory.

Finally, the source contains several visualizations even though the core scientific outputs are also represented numerically in CSV files. The visual products are appropriate for exploratory assignment work, but the underlying report-generation logic could be simplified if the objective were a non-interactive research library rather than a coursework submission.

18 Overall Technical Assessment

The implementation is technically coherent as a self-contained deep-learning assignment. It demonstrates the essential stages of a modern CNN classification experiment: deterministic setup, dataset preparation, augmentation, normalization, structured feature extraction, regularization, optimizer comparison, adaptive training control, class-aware evaluation, inference, and artifact packaging.

Its strongest aspect is the integration of methodological ideas into one executable pipeline. The regularization strategy is not limited to a single mechanism, and the evaluation stage goes beyond accuracy to include macro and weighted class metrics and confusion matrices. The optimizer-learning-rate search is explicit and reproducible within the script, and the best checkpoint is preserved as a concrete model artifact.

From a research-software perspective, the main weaknesses are not failures of the CNN itself but limitations of experimental isolation and software organization. Validation is repeatedly used for development decisions; the sample inference is not independent; the experiment environment is not fully pinned; and the single-file structure mixes concerns. These points matter because a strong academic portfolio should communicate not only that a model can be trained, but that the evaluation protocol and software environment are understood critically.

19 Conclusion

The uploaded source implements a complete five-class flower-image classification workflow around a regularized CNN. Images are downloaded from the TensorFlow example dataset, split into training and validation subsets, cached and prefetched, augmented and normalized within the model, and processed by four convolutional blocks followed by global average pooling and a regularized dense classification head. The script then compares four optimizers at three learning rates, selects a configuration using peak validation accuracy, retrain it with checkpointing and adaptive training controls, evaluates it with multiple multiclass metrics, demonstrates inference, and packages the resulting artifacts.

The source provides enough information to reconstruct the computational method and its major mathematical components without relying on hidden functionality. At the same time, the code does not support stronger claims such as independent test-set generalization, fully deterministic execution across environments, or optimality beyond the tested optimizer-learning-rate grid. The appropriate academic interpretation is therefore that the project demonstrates a coherent, regularized CNN experimentation pipeline with explicit reproducibility measures and a comparatively thorough evaluation stage, while retaining clear opportunities to strengthen experimental isolation, environment capture, and software modularity.

Source Traceability Notes

The analysis in this report is grounded in the uploaded 854-line Python source. Key implementation anchors include global configuration and seed setup (lines 51–78), dataset creation (lines 92–151), model definition (lines 206–370), optimizer search (lines 377–454), full training callbacks (lines 459–502), metric extraction and evaluation (lines 539–645), inference (lines 651–780), assignment-summary generation (lines 789–828), and ZIP packaging (lines 834–854). The source compiles successfully under Python syntax checking. No numerical runtime results are asserted here because the report was required to remain faithful to the source rather than invent or silently reproduce an execution outcome.